

Nested Classes in Java

Last Updated : 26 Jul, 2023

nested classes are different from inherited classes, and follow a structure must like nested if-else or loops

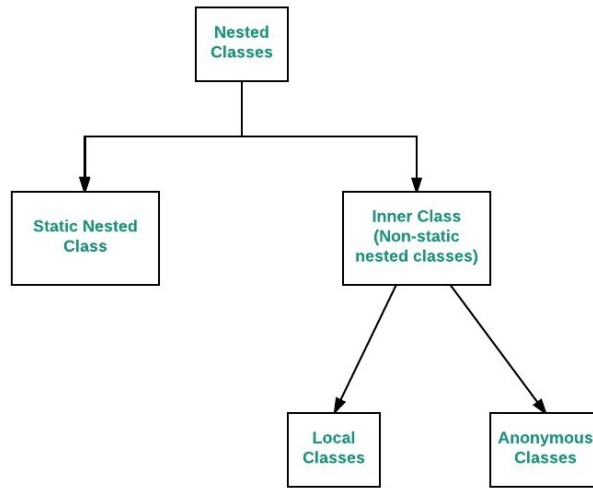


In Java, it is possible to define a class within another class, such classes are known as *nested* classes. They enable you to logically group classes that are only used in one place, thus this increases the use of [encapsulation](#) and creates more readable and maintainable code.

- The scope of a nested class is bounded by **the scope of its enclosing class**. Thus in the below example, the class *NestedClass* does not exist independently of the class *OuterClass*.
- A nested class has access to the members, including private members, **of the class in which it is nested**. But the **enclosing class does not have access to the member of the nested class**.
- A nested class is also **a member of its enclosing class**.
- As a member of its enclosing class, a nested class can be declared *private*, *public*, *protected*, or *package-private*(default).
- Nested classes are divided into two categories:
 1. **static nested class**: Nested classes that are declared *static* are called static nested classes.
 2. **inner class**: An inner class is a non-static nested class.

Syntax:

```
class OuterClass
{
    ...
    class NestedClass
    {
        ...
    }
}
```



Static nested classes

In the case of normal or regular **inner classes**, without an **outer class** object existing, there cannot be an inner class object. i.e., an object of the inner class is always strongly associated with an outer class object. But in the case of static nested class, Without an outer class object existing, there may be a static nested class object. i.e., an object of a static nested class is not strongly associated with the outer class object. As with class methods and variables, a static nested class is associated with its outer class. And like static class methods, a static nested class cannot refer directly to instance variables or methods defined in its enclosing class: it can use them only through an **object reference**. They are accessed using the enclosing class name.

```
OuterClass.StaticNestedClass
```

For example, to create an object for the static nested class, use this syntax:

```
OuterClass.StaticNestedClass nestedObject =  
    new OuterClass.StaticNestedClass();
```

Below is the implementation of the above method:

```
// Java program to demonstrate accessing  
// a static nested class  
  
// outer class  
class OuterClass {
```

```

// static member
static int outer_x = 10;

// instance(non-static) member
int outer_y = 20;

// private member
private static int outer_private = 30;

// static nested class
static class StaticNestedClass {
    void display()
    {
        // can access static member of outer class
        System.out.println("outer_x = " + outer_x);

        // can access private static member of
        // outer class
        System.out.println("outer_private = "
            + outer_private);

        // The following statement will give compilation
        // error as static nested class cannot directly
        // access non-static members
        // System.out.println("outer_y = " + outer_y);

        // Therefore create object of the outer class
        // to access the non-static member
        OuterClass out = new OuterClass();
        System.out.println("outer_y = " + out.outer_y);

    }
}

// Driver class
public class StaticNestedClassDemo {
    public static void main(String[] args)
    {
        // accessing a static nested class
        OuterClass.StaticNestedClass nestedObject
            = new OuterClass.StaticNestedClass();

        nestedObject.display();
    }
}

```

Output

```
outer_x = 10
outer_private = 30
outer_y = 20
```

Inner classes

To instantiate an inner class, you must first instantiate the outer class. Then, create the inner object within the outer object with this syntax:

```
OuterClass.InnerClass innerObject = outerObject.new InnerClass();
```

There are two special kinds of inner classes :

1. [Local inner classes](#)
2. [Anonymous inner classes](#)

```
// Java program to demonstrate accessing
// a inner class

// outer class
class OuterClass {
    // static member
    static int outer_x = 10;

    // instance(non-static) member
    int outer_y = 20;

    // private member
    private int outer_private = 30;

    // inner class
    class InnerClass {
        void display()
        {
            // can access static member of outer class
            System.out.println("outer_x = " + outer_x);
        }
    }
}
```

```

        // can also access non-static member of outer
        // class
        System.out.println("outer_y = " + outer_y);

        // can also access a private member of the outer
        // class
        System.out.println("outer_private = "
                           + outer_private);
    }
}

// Driver class
public class InnerClassDemo {
    public static void main(String[] args)
    {
        // accessing an inner class
        OuterClass outerObject = new OuterClass();

        OuterClass.InnerClass innerObject
            = outerObject.new InnerClass();

        innerObject.display();
    }
}

```

Output

```

outer_x = 10
outer_y = 20
outer_private = 30

```

Comparison between a normal or regular class and a static nested class

S.No.	Normal/Regular inner class	Static nested class
1.	Without an outer class object existing, there cannot be an inner class object. That is, the inner class object is always associated with the outer class object.	Without an outer class object existing, there may be a static nested class object. That is, a static nested class object is not associated with the outer class object.
2.	As the main() method can't be declared, the regular inner class can't be invoked directly from the command prompt.	As the main() method can be declared, the static nested class can be invoked directly from the command prompt.
3.	Both static and non-static members of the outer class can be accessed directly.	Only a static member of an outer class can be accessed directly.

 Comment

More info 

Next Article 

Java.util.Objects class in Java

Similar Reads

Classes and Objects in Java

In Java, classes and objects are basic concepts of Object Oriented Programming (OOPs) that are used to represent real-world concepts and entities. The class represents a group of objects having similar...

 11 min read

Understanding Classes and Objects in Java

The term Object-Oriented explains the concept of organizing the software as a combination of different types of objects that incorporate both data and behavior. Hence, Object-oriented programming(OOPs) is...

 10 min read

Inner Class in Java

In Java, inner class refers to the class that is declared inside class or interface which were mainly introduced, to sum up, same logically relatable classes as Java is object-oriented so bringing it closer to...

🕒 11 min read

Anonymous Inner Class in Java

Nested Classes in Java is prerequisite required before adhering forward to grasp about anonymous Inner class. It is an inner class without a name and for which only a single object is created. An anonymous...

🕒 7 min read

Nested Classes in Java

In Java, it is possible to define a class within another class, such classes are known as nested classes. They enable you to logically group classes that are only used in one place, thus this increases the use of...

🕒 5 min read

Java.util.Objects class in Java

Java 7 has come up with a new class Objects that have 9 static utility methods for operating on objects. These utilities include null-safe methods for computing the hash code of an object, returning a string for...

🕒 7 min read

Different ways to create objects in Java

There are several ways by which we can create objects of a class in java as we all know a class provides the blueprint for objects, you create an object from a class. This concept is under-rated and sometimes...

🕒 6 min read

How are Java objects stored in memory?

In Java, all objects are dynamically allocated on Heap. This is different from C++ where objects can be allocated memory either on Stack or on Heap. In JAVA, when we allocate the object using new(), the...

🕒 3 min read

Passing and Returning Objects in Java

Although Java is strictly passed by value, the precise effect differs between whether a primitive type or a reference type is passed. When we pass a primitive type to a method, it is passed by value. But when w...

🕒 6 min read

Lambda Expression in Java

Lambda expressions in Java, introduced in Java SE 8, represent instances of functional interfaces (interfaces with a single abstract method). They provide a concise way to express instances of single-...

🕒 5 min read

Serialization and Deserialization in Java with Example

Serialization is a mechanism of converting the state of an object into a byte stream. Deserialization is the reverse process where the byte stream is used to recreate the actual Java object in memory. This...

🕒 7 min read

Garbage Collection in Java

Garbage collection in Java is the process by which Java programs perform automatic memory management. Java programs compile to bytecode that can be run on a Java Virtual Machine, or JVM for...

🕒 9 min read

How to prevent objects of a class from Garbage Collection in Java

The garbage collector in Java is automatic, i.e the user doesn't have to manually free an occupied memory which was dynamically allocated. And how does a garbage collector decide which object is to be...

🕒 5 min read

Count number of a class objects created in Java

The idea is to use static member in the class to count objects. A static member is shared by all objects of the class, all static data is initialized to zero when the first object is created if no other initialization is...

🕒 1 min read

Article Tags :

Java

Java-Class and Object

Practice Tags :

Java

Java-Class and Object

